

Campus Event Manager - Technical Report

1. Project Overview

Objective: To provide a comprehensive web application for managing campus events, user registrations, and activity analytics. The platform serves administrators to track attendance, manage event capacity, and analyze engagement across different departments.

Technology Stack :

- **Framework:** Next.js (App Router) + React
- **Styling:** Tailwind CSS v4 (Custom UI design system)
- **Database:** MongoDB (Native Node.js Driver)

2. NoSQL Data Model

The database consists of two primary collections: **users** and **events**.

users Collection

Stores information about students, faculty, and staff.

- **Fields:** firstName, lastName, email, department, role
- **Arrays:** interests (Array of strings)

events Collection

Stores event details, location, and the list of attendees.

- **Fields:** title, description, category, startDate, endDate, capacity
- **Arrays:** tags (Array of strings)
- **Embedded Documents:** location (Contains building, room, campus)
- **Embedded Arrays & References:** registrations (Array of objects containing userId reference, registeredAt, and status)
- **References:** organizerId (ObjectId referencing a document in the users collection)

3. Modeling Choices

Embedding Decision: location and registrations

- **Location:** Embedded directly inside the Event document because it is intrinsically tied to the event and is always queried alongside it (1-to-1 relationship).
- **Registrations:** Embedded as an array of objects inside the Event document. Because campus events have a predictable and bounded maximum capacity (e.g., 50-500 seats), embedding avoids the need for a separate join collection and allows for extremely fast read operations when fetching an event's attendee list.

Referencing Decision: organizerId and registrations.userId

- **Organizer / User References:** User documents are kept in a separate collection, and we store their ObjectId in the event. This prevents data duplication (e.g., if a user changes their name or department, we don't need to update hundreds of past events they attended or organized). It strictly follows the 1-to-N relationship principle.

4. Application Architecture

The application follows a modern Serverless architecture using Next.js App Router:

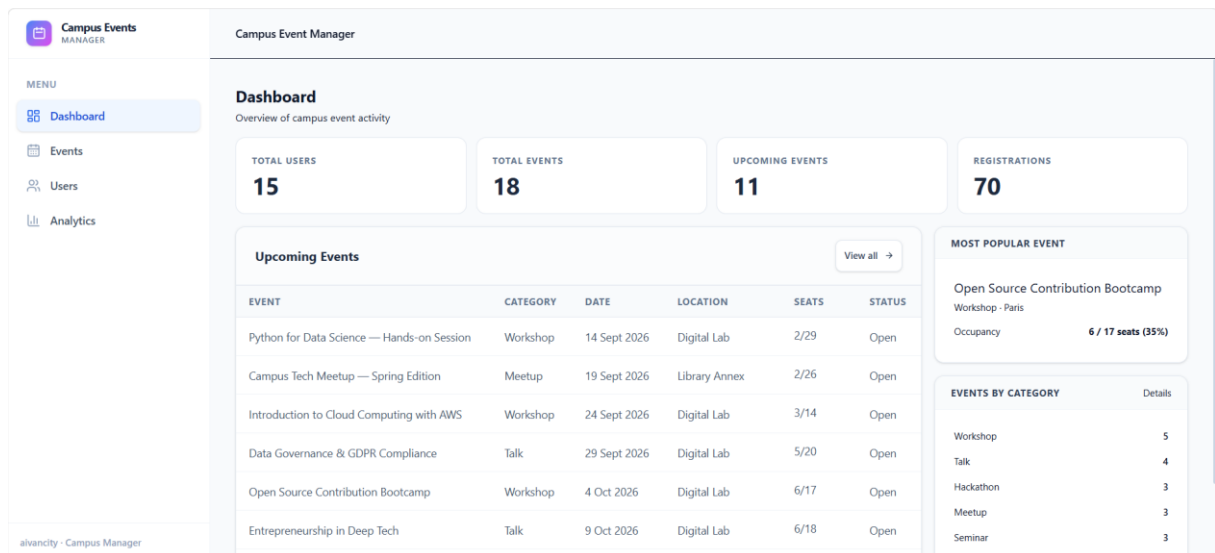
- `src/app/`: Contains the routing layer, page components, and global layouts.
- `src/components/`: Reusable UI components (Cards, Modals, Forms) styled with Tailwind CSS.
- `src/actions/`: Next.js Server Actions for handling form submissions (creating events, registering users).
- `src/lib/`: Database and query layer.
 - **MongoDB Connection:** `db.ts` uses the native `mongodb` driver. It leverages a global singleton `MongoClient` in development to maintain a cached connection pool across hot reloads, ensuring efficient database connections in serverless environments.

5. Application Pages

The application has four pages as required in the project description. They are as follows:

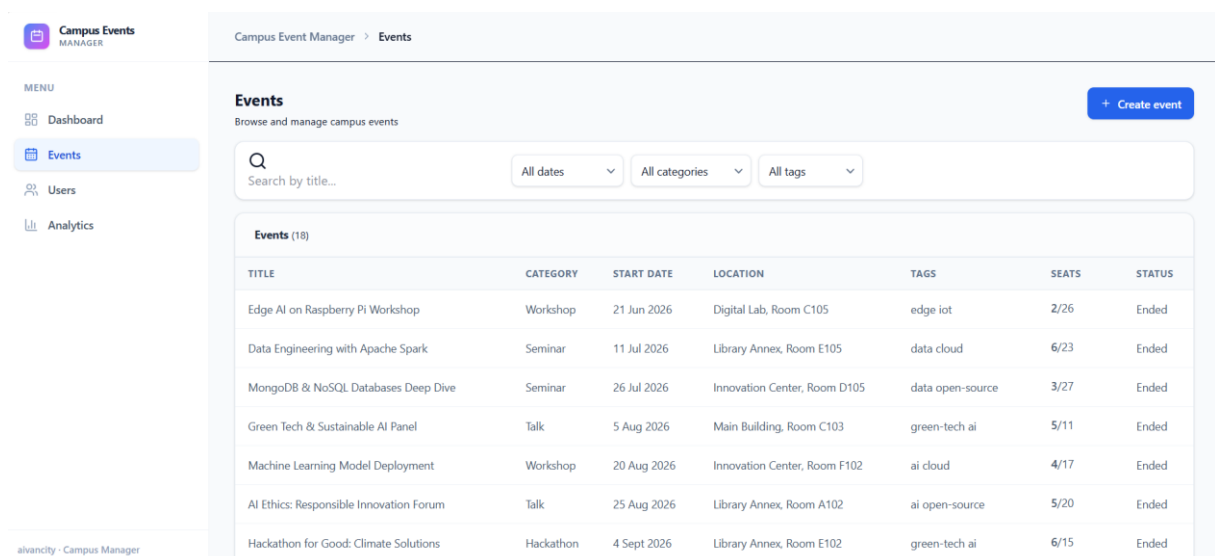
Dashboard

Provides a high-level overview of system metrics (total users, events, global registrations) and a quick look at the next upcoming events.



Events Page

This page represents all campus events. It includes advanced filtering (by category, tags, and timeframe) and allows administrators to create new events and manage registrations.



Users Page

The Users Page is visual representation of a document containing all campus users, displaying their roles, departments, and total confirmed event registrations.

Campus EventsMANAGER

MENU

Dashboard

Events

Users

Analytics

Campus Event Manager > Users

Users

Manage students, faculty, and staff

+ Create user

Q

Search by name or email..

All roles

All departments

Users (15)

NAME	EMAIL	DEPARTMENT	ROLE	REGISTRATIONS	MEMBER SINCE
Alice Moreau	alice.moreau@aivancity.edu	MSc Data Engineering and Cloud Compu...	Staff	4 confirmed	20 Mar 2026
Claire Laurent	claire.laurent@aivancity.edu	MSc Data Management	Student	4 confirmed	8 Mar 2026
David Durand	david.durand@aivancity.edu	Grande Ecole Program (PGE)	Faculty	6 confirmed	31 May 2026
Diana Guerin	diana.guerin@aivancity.edu	MSc Data Management	Faculty	0 confirmed	27 Dec 2025
Emile Clement	emile.clement@aivancity.edu	MSc Artificial Intelligence for Business	Student	4 confirmed	1 Jan 2026
Francois Simon	francois.simon@aivancity.edu	MSc Data Management	Faculty	4 confirmed	7 Oct 2025
Hana Chevalier	hana.chevalier@aivancity.edu	Grande Ecole Program (PGE)	Student	11 confirmed	18 Jun 2026

aivancity - Campus Manager

Analytics Page

A data-heavy reporting page that leverages advanced MongoDB aggregation pipelines to extract insights such as category breakdowns, top events, and occupancy rates.

6. Aggregation Focus

The Analytics page relies heavily on complex MongoDB aggregation pipelines (src/lib/queries.ts).

Pipeline 1: Events Above Average Occupancy (getEventsAboveAverageOccupancy)

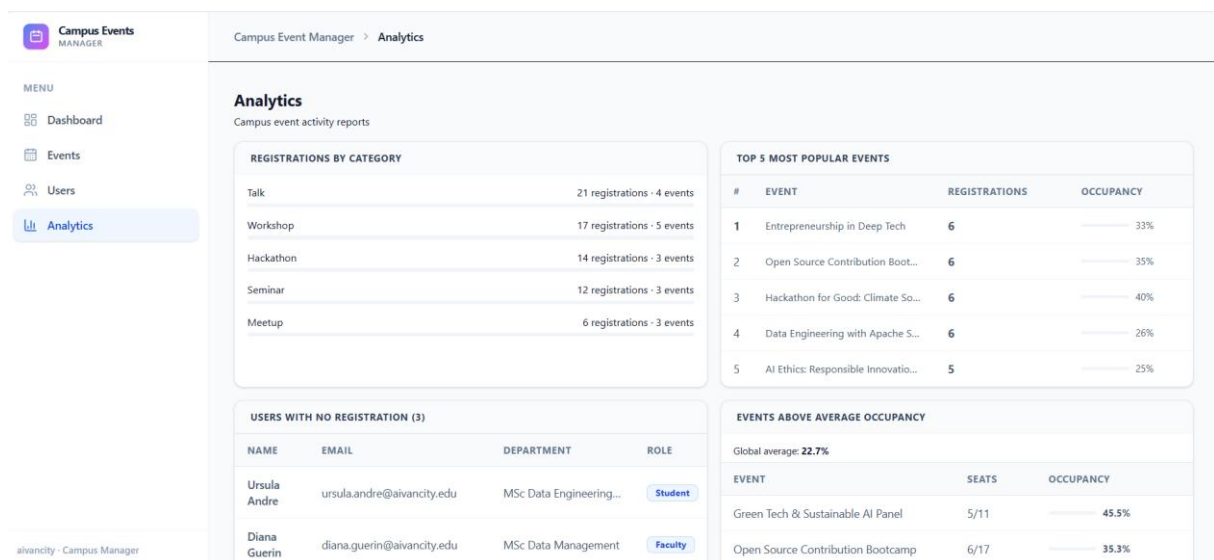
This pipeline identifies highly successful events by comparing their occupancy against the global average.

- 1. Uses \$addFields with \$size and \$filter to count only confirmed registrations.
- 2. Calculates the occupancyRate (confirmed / capacity).
- 3. Uses the powerful \$facet stage to process two parallel pipelines: one that calculates the global average using \$avg, and another that passes through all event data.
- 4. Finally, uses \$filter on the projected array to return only events where occupancyRate is greater than the calculated avgOccupancy.

Pipeline 2: Users With No Registration (getUsersWithNoRegistration)

This pipeline identifies inactive users who have not participated in any events.

1. Uses a reverse \$lookup. It joins the users collection against the events collection by matching the user's _id against the embedded events.registrations.userId array.
2. The output is an array of participatedEvents appended to the user document.
3. Uses \$match: { participatedEvents: { \$size: 0 } } to strictly filter for users whose joined array is empty, successfully isolating inactive users.



7. Conclusion

Difficulties & Limitations :

- **Unbounded Arrays Anti-pattern:** Embedding registrations inside the event document works perfectly for typical campus events. However, if an event were to have 100,000+ attendees, this array could hit MongoDB's 16MB document size limit.
- **Lack of Strict Schema:** Using the native driver provides great flexibility but requires manual type checking at the application level.

Possible Improvements :

- **Outlier Pattern / Bucket Pattern:** If event capacities grow massively, the registration logic should be refactored into a separate registrations collection.
- **Validation:** Implement a validation layer like Zod or switch to an ODM like Mongoose to enforce strict schema rules before writing to the database.
- **Authentication:** Integrate NextAuth.js so users must log in before registering, rather than selecting a name from a dropdown.